

I cannot directly generate or export a .docx file for you to download. However, I have formatted the complete document below into clean, standard text.

You can highlight everything below, copy it, and paste it directly into a blank Microsoft Word document. The headings, bullet points, and tables will automatically retain their structure.

(Note: I have converted the architectural diagram into a text outline, as code-based diagrams do not paste correctly into Word).

Systems Analysis and Design Document

Project Name: Keystone Property Management System

Prepared by: Kamvelihle Athabile Dyantyi

Institution: Rosebank College

Date: August 2026

1. Executive Summary

This document outlines the architecture, functional requirements, and technical specifications for the Keystone Property Management System (PMS). The system is designed to streamline rental operations for landlords while providing a simplified, accessible portal for tenants to handle essential tasks such as payments, maintenance requests, and contract retrieval. The architecture strictly adheres to robust security practices and integrates modern mobile development standards, including offline-first capabilities and multi-language support.

2. System Architecture & Technology Stack

The system follows a decoupled, client-server architecture utilizing cost-effective and highly available infrastructure.

- **Frontend (Admin Dashboard):** A responsive web application deployed via Netlify.
- **Frontend (Tenant App):** A native Android application developed in Kotlin.
- **Backend API:** A RESTful API built in C# (ASP.NET Core), serving as the communication layer between the frontends and the database.
- **Database:** A relational SQL database hosted on an Oracle Cloud Infrastructure (OCI) Always Free tier instance.
- **Object Storage:** Cloudflare R2 or Azure Blob Storage for securely storing PDF lease agreements, Proof of Payment (POP) image uploads, and emergency report photos.

2.1 Portfolio of Evidence (PoE) Integration Stack

- **Authentication (SSO):** Firebase Authentication.
- **Settings Management:** Android Preferences DataStore.
- **Offline Mode with Sync:** RoomDB (local SQLite) + WorkManager.
- **Real-time Notifications:** Firebase Cloud Messaging (FCM).
- **Multi-language Support:** Android String Resources (res/values-b+zu for isiZulu, res/values-b+af for Afrikaans).

3. Role-Based Access Control (RBAC)

The application implements strict access controls to ensure data privacy and system integrity.

- **Administrator (Landlord):** Has full system access. Responsible for creating tenant

profiles, assigning properties/units, and generating secure invite credentials.

- **Tenant:** Access is restricted strictly to data associated with their assigned unit. Registration is invite-only to prevent unauthorized sign-ups. Initial login utilizes Single Sign-On (SSO) via Firebase.

4. Functional Requirements & Workflows

4.1 Tenant Modules (Kotlin Android App)

- **Payment Hub:** Form to upload POPs, declare the amount paid, and view personal payment history and status (Pending / Verified).
- **Maintenance Portal:** Ticketing interface to report routine issues, supporting photographic evidence.
- **Document Vault:** Read-only access to download or view their active lease agreement.
- **Digital Noticeboard:** Broadcast channel for property-wide announcements from the landlord.
- **Emergency Report (Offline-First):** A dedicated submission form where the tenant describes the emergency and attaches photos directly within the app. If the device is offline, RoomDB stores the data. Upon reconnection, WorkManager syncs the ticket to the C# API. Quick-dial shortcuts for Police, Ambulance, and Plumber remain accessible on the same screen.
- **App Settings:** Tenants can toggle UI languages (English, isiZulu, Afrikaans) and notification preferences.

4.2 Landlord Modules (Web Admin Dashboard)

- **Financial Dashboard:** Visualizes cash flow and categorizes expenditures, including provisional, tenant-reported amounts pending confirmation.
- **Property & Tenant Management:** Hierarchical grouping of properties, units, and active tenants.
- **Ledger System:** Interface to verify tenant POP submissions and track rental arrears.
- **Maintenance Kanban & Emergency Viewer:** A ticket tracking board for managing tenant complaints and repairs. Emergency tickets present the full written description, submission timestamp, and an inline photo viewer showing all attached images at full resolution.

4.3 Automated Workflows

- **Payment Verification:** When a tenant uploads a POP, they declare the amount paid. This immediately reflects on the landlord's dashboard as a provisional entry. The system sends the landlord an automated FCM notification showing who submitted it, the unit, the amount declared, and a link to the file. The status remains "Pending" until the landlord manually verifies the evidence.
- **Emergency Alerts:** Attached emergency photos are uploaded securely to object storage via the backend API. Once saved, the system triggers an automatic server-side push notification to the landlord.

5. Database Schema (Data Dictionary)

Entity	Fields	Description
Users	UserId (UUID), FirebaseAuthUid, FullName, Email, Role, AssignedUnitId	Stores all Landlord and Tenant profiles. Linked securely to Firebase SSO.
Properties	UnitId (UUID), PropertyName, UnitNumber, MonthlyRent, LeaseUrl	Defines the hierarchical structure of the rental units and stores secure blob links to contracts.
Payments	PaymentId (UUID), TenantId, DeclaredAmount, PopImageUrl, Status	The financial ledger handling pending and verified POP submissions.
Tickets	TicketId (UUID), TenantId, Category, Description, PhotoUrls, State	Stores routine maintenance and emergency reports, synced from the tenant's RoomDB.

6. REST API Specification (C# ASP.NET Core)

- **Authentication Gateway:** Middleware validates the Firebase JWT attached to the Authorization header of every incoming request.
- **POST /api/v1/payments:** Accepts multipart form data (declared amount and POP file). Uploads the file to object storage, creates a pending database record, and fires an FCM alert to the landlord.
- **POST /api/v1/tickets/sync:** Receives JSON payloads from the tenant's offline WorkManager queue, persisting synced tickets to the SQL Server and triggering emergency alerts.
- **GET /api/v1/properties/my-lease:** Generates and returns a time-limited signed URL from Azure Blob/Cloudflare R2, allowing secure, temporary access to the PDF.

7. Infrastructure & Cost Optimization

To accommodate budget constraints without sacrificing reliability, the infrastructure is heavily optimized for zero or near-zero operational costs. By leveraging Oracle Cloud's Always Free tier for backend compute and database hosting, coupled with Netlify for static frontend delivery, monthly recurring costs are effectively neutralized. Storage costs for POPs, lease contracts, and emergency photos via Cloudflare R2 will remain comfortably within free tier limits given the expected upload volumes.

8. Security Considerations

- **Authentication & Session Integrity:** All tenant API calls are validated server-side using secure SSO JWTs.
- **File Authorization:** The backend API must validate the active user session before serving private documents or photos from blob storage. Emergency photos are served via time-limited signed URLs so they cannot be shared or hotlinked externally.
- **Data Validation:** Server-side sanitization of all tenant inputs, particularly file uploads (restricting to PDF/JPEG/PNG formats and enforcing a maximum file size per upload).
- **Notification Integrity:** All landlord notifications are triggered server-side upon confirmed, authenticated file storage to prevent spoofed or dropped alerts.